

Token Diet: Benchmarking LLM Token Optimization Strategies

Every LLM API call is billed on two meters — **input tokens** (prompt + data) and **output tokens** (the response) — and output tokens are usually priced several times higher. For a pipeline that enriches records at scale, small per-call inefficiencies compound fast.

This repo benchmarks a naive baseline against six token-optimization strategies, all run on the **same 50 records, same model, same infrastructure**, so the comparison is apples-to-apples. Each strategy logs real input/output token counts and its sentiment label is diffed against the baseline's label to measure accuracy trade-offs, not just cost.

Result: the best strategy cut total token usage by **45%** while still agreeing with the baseline's sentiment labels on **92%** of reviews. Full write-up with analysis: [**Medium article link**]

Dataset

Reviews are a 50-row subset of a public e-commerce food reviews dataset ([source](#), Kaggle, via Stanford SNAP) — ~568K reviews, 1999–2012.

If citing per the dataset's request:

J. McAuley and J. Leskovec. *"From amateurs to connoisseurs: modeling the evolution of user expertise through online reviews."* WWW, 2013.

The subset (`reviews_subset_50.csv`) is not included in this repo — download the full dataset from Kaggle and sample your own subset, or use `utilities.py`'s `load_input_dataset()` against any CSV with `Id` and `Text` columns.

Strategies Benchmarked

#	Strategy	Notebook	What it changes vs. baseline
0	Baseline	<code>Baseline_Token_Usage_Experiment.py</code>	One call per review, open-ended prompt
1	Prompt Optimization	<code>Prompt_Optimization.py</code>	Minimal instruction, forces one-word answer
2	Batch Processing	<code>Batch_Processing_Token_Optimization.py</code>	5 reviews per API call, structured output
3	Binary Classification	<code>Binary_Classification.py</code>	Collapses to 2 labels, <code>max_tokens=10</code>
4	Preprocessing & Filtering	<code>Preprocessing_and_Filtering.py</code>	Cleans/truncates review text before prompting
5	Dynamic Prompt Construction	<code>Dynamic_Prompt_Construction.py</code>	Prompt length varies by review length
6	Few-Shot Sentiment Intensity	<code>Few-Shot_Sentiment_Intensity.py</code>	Examples instead of instructions, 1-5 scale

`utilities.py` holds the shared helpers: client setup, sentiment normalization, results saving, and summary printing, imported by every notebook via `%run`.

Results Summary

Strategy	Total Tokens	Token Savings vs Baseline	Agreement with Baseline
Baseline	12,411	—	100% (reference)
Prompt Optimization	6,816	45.1%	92.0% (46/50)
Batch Processing	7,170	42.2%	88.0% (44/50)
Binary Classification	7,276	41.4%	84.0% (42/50)
Preprocessing & Filtering	11,241	9.4%	96.0% (48/50)
Few-Shot Intensity	11,326	8.7%	40.0% (20/50)
Dynamic Prompt	11,720	5.6%	68.0% (34/50)

Takeaway: techniques that constrain what the model is *allowed to output* (short instructions, low `max_tokens`, structured batch responses) delivered the biggest, safest wins. Techniques that only changed the *input* side without capping output (dynamic prompts, few-shot examples) barely moved the needle on cost and sometimes hurt accuracy.

Running These Notebooks

These are Databricks notebooks (`(# Databricks notebook source)` format) — import them directly into a Databricks workspace, or adapt the `dbutils.widgets` / `%run` calls for local Jupyter use.

1. Set up your API key as a secret (never hardcode it)

```
bash

databricks secrets create-scope token_strategy_comparison
databricks secrets put-secret token_strategy_comparison anthropic_api_key
```

See `README_SECRETS_SETUP.md` for details.

2. Set your input/output paths

Each notebook takes `input_dataset_path` and `output_folder_path` as job widgets — set these to your own workspace paths (replace `<your-databricks-username>` in the defaults).

3. Run in order

```
Baseline_Token_Usage_Experiment.py      # run first – other strategies
compare against this
Prompt_Optimization.py
Batch_Processing_Token_Optimization.py
Binary_Classification.py
Preprocessing_and_Filtering.py
Dynamic_Prompt_Construction.py
Few-Shot_Sentiment_Intensity.py
```

Each notebook saves two CSVs to `output_folder_path`: `*_raw_results.csv` (includes the model's raw, unprocessed response) and `*_normalized_results.csv` (cleaned into one of four labels: Positive, Negative, Mixed, Neutral).

Model & Pricing

Notebooks default to `claude-haiku-4-5-20251001`. Pricing constants are set per-notebook — check [Anthropic's pricing page](#) for current rates before running at any real volume, as prices are subject to change.

Repo Structure

```
.
├── README.md                    # this file
├── README_SECRETS_SETUP.md     # API key / secret scope
├── setup
│   ├── utilities.py            # shared helpers (imported
│   │   │   via %run)
│   ├── Baseline_Token_Usage_Experiment.py
│   ├── Prompt_Optimization.py
│   ├── Batch_Processing_Token_Optimization.py
│   ├── Binary_Classification.py
│   ├── Preprocessing_and_Filtering.py
│   ├── Dynamic_Prompt_Construction.py
│   └── Few-Shot_Sentiment_Intensity.py
```

License

MIT — use, adapt, and benchmark your own strategies freely.